

Amazon Web Services

Security, Identity & Compliance Complete Learning Guide

Chapter 1 — AWS Identity and Access Management (IAM)

Chapter 2 — AWS Key Management Service (KMS)

Chapter 3 — Amazon Macie

Chapter 4 — AWS Secrets Manager

Chapter 5 — AWS Shield

Chapter 6 — AWS WAF (Web Application Firewall)

Introduction | Core Concepts | Architecture | Features | Use Cases | Integrations | Comparisons

Beginner-Friendly • In-Depth • Practical

Table of Contents

Ch.	Service	One-Line Description
1	AWS IAM	Control WHO can do WHAT on WHICH AWS resource — the foundation of all AWS security
2	AWS KMS	Create and manage cryptographic keys that encrypt data across all AWS services
3	Amazon Macie	AI-powered sensitive data discovery and classification in Amazon S3
4	AWS Secrets Manager	Securely store, rotate, and retrieve database credentials, API keys, and secrets
5	AWS Shield	Managed DDoS protection — Standard (free) and Advanced (enterprise-grade)
6	AWS WAF	Web Application Firewall — filter malicious HTTP/HTTPS traffic before it hits your app

Each chapter covers: Simple Introduction • Real-World Analogy • Core Concepts and Terminology • Architecture and Workflow • Key Features • Real-World Use Cases by Industry • AWS Service Integrations • Comparison Tables.

Chapter 1 — AWS Identity and Access Management (IAM)

Control Who Can Do What on Which AWS Resource

1. Simple Introduction

AWS IAM (Identity and Access Management) is a free AWS service that enables you to manage access to AWS services and resources securely. IAM controls AUTHENTICATION (who are you?) and AUTHORIZATION (what are you allowed to do?). It is the single most important security service in AWS — every API call to every AWS service is checked against IAM policies before being allowed or denied.

Analogy — The Corporate Badge and Door Lock System

Imagine an office building with hundreds of rooms (AWS services and resources). IAM is the badge-access system. Each employee (IAM user/role) gets a badge (credentials). The system checks the badge at every door: Is this badge valid? Does this employee have permission to enter THIS room? Security admins (IAM admins) control exactly which rooms each employee can enter, and they can revoke access instantly. Guests (federated users) get temporary visitor badges (STS tokens) that expire automatically.

Business Problem IAM Solves

Multiple people and services need different levels of access to AWS — not everyone should have full access

Applications running on EC2/Lambda need to call AWS APIs without hardcoding passwords

Enforce least-privilege: grant only the minimum permissions needed for each task

Audit trail: who did what and when — every API call logged in CloudTrail

Enable single sign-on (SSO) and federation with corporate identity providers (Active Directory, Okta)

2. Core Concepts & Terminology

Term	Definition	Example
IAM User	A permanent identity (person or application) with long-term credentials (username/password + access keys)	developer@company.com, ci-deploy-user
IAM Group	A collection of IAM users. Attach policies to the group; all members inherit permissions	Developers group, DBAdmins group
IAM Role	A temporary identity with permissions — assumed by users, services, or external accounts. No long-term credentials	EC2InstanceRole, LambdaExecutionRole, CrossAccountRole
IAM Policy	A JSON document defining permissions: which Actions on which Resources under which Conditions	Allow s3:GetObject on arn:aws:s3:::my-bucket/*
Managed Policy	A standalone policy created and managed by AWS (AWS Managed) or by you (Customer Managed). Can be attached to multiple identities	AmazonS3ReadOnlyAccess, MyCustomPolicy

Term	Definition	Example
Inline Policy	A policy embedded directly in a single user/group/role. Deleted when that identity is deleted	One-off policy embedded in a specific role
Permission Boundary	An advanced feature that sets the MAXIMUM permissions an identity can have — even if other policies grant more	Developers can only work within the dev boundary
STS (Security Token Service)	AWS service that issues temporary, limited-privilege credentials (Access Key + Secret Key + Session Token + Expiry)	AssumeRole returns STS credentials valid for 1 hour
AssumeRole	The STS API call to obtain temporary credentials for a role	Lambda assumes S3AccessRole to read a bucket
Trust Policy	JSON policy on a ROLE that defines WHO is allowed to assume it	Allow ec2.amazonaws.com to assume this role
Principal	The entity making a request: IAM user, role, AWS service, or external account	arn:aws:iam::123456789012:user/alice
ARN (Amazon Resource Name)	Unique identifier for any AWS resource	arn:aws:iam::123456789012:role/MyRole
MFA (Multi-Factor Authentication)	Require a second factor (authenticator app or hardware key) for login or sensitive actions	Enforce MFA for s3:DeleteBucket via condition
Access Analyzer	Identifies resources (S3 buckets, IAM roles) that are shared with external entities	Alert when S3 bucket becomes publicly accessible
Service Control Policy (SCP)	AWS Organizations feature — guardrails that restrict what member accounts can do, regardless of their IAM policies	Block all non-approved regions across the org
ABAC (Attribute-Based Access Control)	Control access based on tags on both the IAM principal and the resource	Allow access to resources tagged Env=dev only
RBAC (Role-Based Access Control)	Control access based on named roles attached to users/groups	Developers role has s3:PutObject permission

IAM Policy Structure — JSON Breakdown

IAM Policy JSON — Fully Explained

```
{
  "Version": "2012-10-17", <- Always use this version
  "Statement": [ <- List of permission statements
  {
    "Sid": "AllowS3ReadMyBucket", <- Optional statement ID
    "Effect": "Allow", <- Allow or Deny
    "Principal": "*", <- WHO (omit for identity policies)
    "Action": [ <- WHAT operations are allowed/denied
      "s3:GetObject",
      "s3:ListBucket"
    ],
    "Resource": [ <- ON WHICH resource
      "arn:aws:s3:::my-bucket",
      "arn:aws:s3:::my-bucket/*"
    ],
    "Condition": { <- WHEN this statement applies
      "Bool": {
        "aws:MultiFactorAuthPresent": "true"
      }
    }
  }
]
```

DENY always wins over ALLOW. Default = implicit DENY if no Allow matches.

3. Architecture & How IAM Works

IAM Authorization Flow — Every AWS API Call

1. PRINCIPAL makes a request:

Alice (IAM User) calls s3:PutObject on arn:aws:s3:::prod-bucket/report.csv

2. AWS AUTHENTICATION check:

Are the credentials (access key + secret) valid and not expired? -> Yes -> continue

3. AWS AUTHORIZATION evaluation (policy evaluation logic):

a. Is there an explicit DENY in any applicable policy? -> Yes = DENY immediately

b. Is there an AWS Organizations SCP that denies? -> Yes = DENY immediately

c. Is there an explicit ALLOW in an identity policy? -> Yes -> check resource policy

d. Is there a resource-based policy that allows? -> Combined with identity policy

e. Does a permission boundary restrict it? -> Must be allowed in boundary too

f. Is it a session policy? -> Must also allow

g. No explicit Allow found anywhere -> implicit DENY

4. REQUEST EXECUTED if all checks pass

5. ACTION LOGGED in AWS CloudTrail (who, what, when, where, result)

KEY RULE: Explicit DENY > Explicit ALLOW > Implicit DENY

IAM Identity Types — When to Use Which

Identity	Credentials	Lifetime	Use Case
IAM User	Password + Access Keys	Permanent	Human users who sign into console; legacy service accounts
IAM Group	None (inherits from policies)	Permanent	Organise users by job function; attach policies once
IAM Role (AWS Service)	Temp STS tokens (auto-rotated)	Temporary	EC2, Lambda, ECS, EKS needing AWS API access
IAM Role (Cross-Account)	Temp STS tokens via AssumeRole	Temporary	Account A accesses Account B resources
IAM Role (Federated / SSO)	Temp STS tokens via SAML/OIDC	Temporary	Corporate AD, Google, GitHub Actions CI/CD
Root Account	Email + password	Permanent	Billing, account closure ONLY — never use for daily work

4. Key Features

Feature	Description	Business Value
Free Service	IAM has no additional cost — no limits on users, roles, or policies	Zero cost for access management
Granular Permissions	Control down to individual API actions on specific resources with conditions	Least-privilege security
IAM Roles for Services	EC2/Lambda/ECS assume roles — no hardcoded credentials	Eliminates credential exposure in code

Feature	Description	Business Value
Multi-Factor Authentication	Virtual MFA (Google Authenticator), hardware keys (YubiKey), SMS	Protect accounts from password theft
Federated Identity	SAML 2.0, OIDC, AWS SSO, Cognito — use existing identity providers	Single sign-on for thousands of users
IAM Access Analyzer	Continuously monitors for overly permissive resource policies	Detect unintended public exposure
Credential Report	Download CSV of all users and credential status	Security audit and compliance
Password Policy	Enforce minimum length, complexity, rotation for IAM users	Password security standards
Permission Boundaries	Limit the maximum permissions even admins can grant to users	Delegated administration safely
Service Control Policies	Org-wide guardrails via AWS Organizations	Governance across all accounts
ABAC (Tag-based access)	Use resource and principal tags in conditions	Dynamic, scalable access control
Access Advisor	Shows last accessed time for each service permission	Remove unused permissions

5. Real-World Use Cases

Scenario	IAM Configuration	Key IAM Feature
Developer Team Access	Developers group with PowerUserAccess; DBAdmins group with RDS full access; ReadOnly group for auditors	Groups + Managed Policies
EC2 App Server	EC2 instance profile with role allowing read from S3 and write to DynamoDB	Instance Profile / IAM Role
CI/CD Pipeline (GitHub Actions)	OIDC trust policy on deployment role; GitHub Actions assumes role via web identity token	OIDC Federation
Cross-Account Deployment	Role in prod account trusts dev account; devs assume role to deploy only via approved pipeline	Cross-Account AssumeRole
Banking Compliance	SCPs block all non-approved regions; permission boundaries limit developer blast radius; MFA required for all	SCP + Permission Boundary + MFA
Serverless App	Lambda execution role with least-privilege: only the DynamoDB table and S3 bucket it needs	Lambda Execution Role
Startup with SSO	AWS SSO with Google Workspace OIDC; developers get time-limited roles automatically	Identity Federation / SSO
Data Lake Security	ABAC: analysts can only access datasets tagged with their department tag	Tag-Based Conditions (ABAC)

6. AWS Service Integrations

AWS Service	Integration	Use Case
AWS CloudTrail	Every IAM API call and resource API call logged with identity info	Audit: who did what and when
AWS Organizations + SCPs	SCPs enforce IAM guardrails across all accounts	Enterprise governance

AWS Service	Integration	Use Case
AWS SSO / IAM Identity Center	Central SSO portal using IAM roles under the hood	Single login for all accounts
Amazon Cognito	User pools for app users; identity pools give AWS credentials via IAM roles	App-level identity federation
AWS STS	Issues temporary credentials when roles are assumed	Temporary access for services and federation
AWS KMS	IAM controls who can use KMS keys via key policies and IAM policies	Encryption key access control
Amazon S3	Bucket policies use IAM principals; IAM policies control S3 actions	Object-level access control
AWS Config	Detect and alert on IAM misconfigurations (root MFA off, unused keys)	Compliance monitoring
Amazon GuardDuty	Detects anomalous IAM credential use (e.g. credentials used from unusual location)	Threat detection

7. IAM Best Practices

Practice	Why It Matters	How
Never use root account for daily work	Root has unlimited power — one compromise = full account loss	Create IAM users/roles immediately after account creation
Enable MFA on root and all human users	Passwords can be stolen; MFA prevents unauthorized access	Use virtual MFA app or hardware key
Use roles, not access keys for services	Access keys can be accidentally committed to GitHub	Assign instance profiles to EC2, execution roles to Lambda
Apply least privilege	Overly broad permissions = larger blast radius on compromise	Start with deny-all, add only needed actions
Rotate access keys regularly	Old keys may be compromised without your knowledge	Use Access Advisor to find and remove unused keys
Use AWS Organizations + SCPs	Individual account IAM can still make mistakes; SCPs are a safety net	Block dangerous actions org-wide
Enable CloudTrail in all regions	Without audit logs you cannot investigate incidents	Multi-region CloudTrail to S3 with integrity validation
Use IAM Access Analyzer	Automatically detects when you accidentally make resources public	Enable in every region; review findings weekly

Chapter 2 — AWS Key Management Service (KMS)

Create, Manage & Control Cryptographic Keys for Data Encryption

1. Simple Introduction

AWS KMS (Key Management Service) is a fully managed service that enables you to create and control the cryptographic keys used to encrypt your data. It integrates with most AWS services (S3, RDS, EBS, DynamoDB, etc.) so you can encrypt data at rest with a single checkbox — and KMS manages all the key material, hardware security, and rotation automatically.

Encryption protects data by transforming it into an unreadable format. A cryptographic key is the secret piece of information required to encrypt or decrypt data. Managing keys securely is one of the hardest problems in security — KMS solves it.

Analogy — The Bank Vault Key Service

Imagine you have important documents (data) you need to lock in a vault. AWS KMS is a highly secure locksmith service. It creates unique, tamper-proof keys for your vaults, stores them in a fortified facility (FIPS 140-2 Level 3 HSMS), and only unlocks your vault when you present valid credentials AND you are on the approved list. KMS never hands you the physical master key — it performs the locking/unlocking on your behalf, logging every access in an audit trail.

Business Problem KMS Solves

Encrypt sensitive data at rest (S3, RDS, EBS, DynamoDB) without managing key infrastructure

Centralise key management and access control — one place to control all encryption

Meet compliance requirements (HIPAA, PCI-DSS, GDPR, SOC2) requiring encryption at rest

Audit who accessed which key and when — full CloudTrail integration

Automatically rotate encryption keys annually without re-encrypting data

2. Core Concepts & Terminology

Term	Definition	Detail
KMS Key (CMK)	Customer Master Key — the primary resource in KMS. Represents an encryption key. Never leaves KMS in plaintext	256-bit AES-GCM symmetric by default
Symmetric Key	Same key used for encryption and decryption. Used by all AWS services	AES-256. Most common for AWS service encryption
Asymmetric Key	Public/private key pair. Public key encrypts; private key decrypts. Or used for signing	RSA 2048/3072/4096, ECC P-256/P-384
HMAC Key	Used for generating and verifying Hash-based Message Authentication Codes	Message integrity verification

Term	Definition	Detail
Data Key	A plaintext AES key generated by KMS using a CMK. Used to encrypt actual data (envelope encryption)	GenerateDataKey API call
Envelope Encryption	Encrypt data with a data key; encrypt the data key with a CMK. Only the encrypted data key is stored alongside data	KMS never stores data keys
Key Policy	Resource-based policy attached to a KMS key controlling who can use and administer it	Separate from IAM policies — must explicitly allow access
Key Alias	A friendly name for a KMS key (e.g. alias/myapp-db-key)	Decouple key rotation from application config
AWS Managed Key	Keys created and managed by AWS on your behalf for a specific service. You cannot change their policy	alias/aws/s3, alias/aws/rds — free to use
Customer Managed Key (CMK)	Keys you create, own, and manage. You control rotation, policy, and deletion	\$1/month per key + API call costs
AWS Owned Key	Keys owned entirely by AWS, shared across accounts for internal service use. Invisible to customers	Some DynamoDB encryption options
Key Rotation	Automatically generate new key material every year. Old material kept to decrypt old data. Applications use same key ID	One checkbox — no re-encryption of data needed
Key Material	The actual cryptographic bits of the key. Can be AWS-generated or imported (Bring Your Own Key — BYOK)	External key material for regulatory requirements
CloudHSM Key Store	Store KMS key material in your own CloudHSM cluster instead of KMS HSMs	Single-tenant HSM requirement
Grant	Temporary delegated permission to use a KMS key — more flexible than key policies for dynamic access	AWS services create grants automatically when encrypting
Key Deletion	KMS keys can be scheduled for deletion (7-30 day waiting period). IRREVERSIBLE — all data encrypted with it is permanently unrecoverable	Always disable first; test before deleting

Envelope Encryption — How KMS Actually Encrypts Data

Envelope Encryption Flow

WHY: KMS CMK can only encrypt data up to 4 KB directly.

For large data (files, database rows), envelope encryption is used.

ENCRYPTION FLOW:

1. App calls KMS: `GenerateDataKey(KeyId='alias/my-key')`
2. KMS returns:
 - Plaintext data key (AES-256, 32 bytes)
 - Encrypted data key (encrypted under the CMK, stored with data)
3. App encrypts the actual data using the plaintext data key
4. App DISCARDS the plaintext data key from memory
5. App stores: [Encrypted Data] + [Encrypted Data Key]

DECRYPTION FLOW:

1. App sends encrypted data key to KMS: `Decrypt(CiphertextBlob=...)`
2. KMS checks IAM + key policy -> authorised -> decrypts the data key
3. KMS returns plaintext data key
4. App decrypts the data using the plaintext data key
5. App discards plaintext data key from memory

RESULT: Actual data never touches KMS. CMK never leaves KMS HSM.

Attacker with stolen data file cannot decrypt without KMS access.

3. Key Types — When to Use Which

Key Type	Algorithm	Operations	Use Case
Symmetric (SYMMETRIC_DEFAULT)	AES-256-GCM	Encrypt, Decrypt, GenerateDataKey	All AWS service encryption, app-level encryption
RSA 2048 / 3072 / 4096	RSA-OAEP	Encrypt/Decrypt or Sign/Verify	Envelope encryption in non-AWS SDKs; digital signatures
ECC P-256 / P-384	ECDSA	Sign/Verify only	Code signing, JWT signing, IoT device auth
SM2 (China regions only)	SM2	Encrypt/Decrypt, Sign/Verify	China regulatory compliance
HMAC	HMAC-SHA256/384/512	GenerateMac, VerifyMac	Message integrity, API request signing

4. Key Features

Feature	Description	Business Value
FIPS 140-2 Level 3 HSMs	Key material stored in validated hardware security modules	Tamper-proof key storage
Automatic Key Rotation	Rotate key material annually with one checkbox	Compliance without operational overhead
CloudTrail Integration	Every KMS API call logged: who, when, which key, which operation	Complete encryption audit trail

Feature	Description	Business Value
Multi-Region Keys	Replicate KMS key to other regions for cross-region encryption	Encrypt in us-east-1, decrypt in eu-west-1
Key Policies + IAM	Two-layer access control: key policy + IAM policy must both allow	Defence-in-depth for key access
Grants	Temporary, programmable delegated key access — no policy updates needed	Dynamic access for AWS services
Custom Key Store (CloudHSM)	Store key material in customer-owned CloudHSM cluster	Single-tenant HSM requirement
BYOK (Bring Your Own Key)	Import your own key material into KMS	External key material for regulatory control
Asymmetric Key Support	RSA and ECC keys for signing and public-key encryption	Digital signatures, PKI
Shared & Dedicated HSM	Default = shared AWS HSMs; CloudHSM key store = dedicated	Flexibility between cost and isolation
Cross-Account Key Sharing	Allow other accounts to use your KMS key via key policy	Centralised encryption key management
Deletion Protection	7-30 day waiting period before key deletion	Prevent accidental unrecoverable data loss

5. Real-World Use Cases

Industry	Use Case	KMS Feature Used
Healthcare (HIPAA)	Encrypt patient records in S3 and RDS with customer-managed keys; audit every decryption in CloudTrail	CMK + CloudTrail + Key Policy
Banking (PCI-DSS)	Encrypt cardholder data with CMK; separate key per environment (dev/staging/prod); automatic annual rotation	Per-env CMK + Key Rotation
SaaS Multi-Tenant	Each customer gets their own CMK; their data can only be decrypted with their key; revoke access by disabling key	Per-tenant CMK + Key Policy
Code Signing	Sign container images with asymmetric ECC key; verify signatures before deployment to EKS	Asymmetric Key + Sign/Verify
Cross-Region DR	Multi-region CMK: encrypt data in us-east-1, replicate to eu-west-1 and decrypt there for DR	Multi-Region Keys
Regulatory BYOK	Bank imports its own HSM-generated key material into KMS to maintain key custody proof	BYOK (Import Key Material)
Lambda Encryption	Lambda function encrypts sensitive config variables and S3 outputs with a CMK	S3 SSE-KMS + Lambda IAM Role
Database Encryption	RDS PostgreSQL with CMK; DBA role can use key but cannot delete it; auditors see all access	RDS CMK + Key Policy separation

6. AWS Service Integrations

AWS Service	How KMS Integrates	Encryption Type
Amazon S3	SSE-KMS: objects encrypted with CMK. Bucket key reduces KMS API calls by 99%	Server-side encryption
Amazon RDS / Aurora	Encrypt DB instance and snapshots with CMK at creation	Storage-level encryption
Amazon EBS	Encrypt EBS volumes with CMK; all snapshots and copies inherit encryption	Block storage encryption
AWS Secrets Manager	Secrets encrypted with CMK at rest	Secrets at-rest encryption
Amazon DynamoDB	Table encryption with CMK (or AWS owned key)	Table-level encryption
Amazon SNS / SQS	Message encryption with CMK for sensitive event payloads	Message-level encryption
AWS Lambda	Encrypt environment variables with CMK	Config encryption
Amazon CloudWatch Logs	Encrypt log groups with CMK	Log data encryption
AWS Systems Manager	Parameter Store SecureString parameters encrypted with CMK	Parameter encryption
AWS IAM	IAM policies control who can call KMS APIs (kms:Decrypt, kms:GenerateDataKey)	Access control layer
AWS CloudTrail	Log all KMS API calls for audit and forensics	Audit trail

7. KMS Key Types Comparison

Dimension	AWS Owned Key	AWS Managed Key	Customer Managed Key
Who creates	AWS	AWS (for you)	You
Who manages	AWS	AWS	You
Cost	Free	Free to use	\$1/month + API calls
Visibility	Not visible	Visible, not editable	Fully visible + editable
Key Policy	AWS controls	Limited control	Full control
Rotation	AWS manages	Auto every year	Optional auto or manual
BYOK	No	No	Yes (import material)
CloudHSM	No	No	Yes (custom key store)
Best For	Default service encryption where you don't need control	Services needing audit trail without custom policy	Compliance, multi-tenant, BYOK, audit requirements

Chapter 3 — Amazon Macie

AI-Powered Sensitive Data Discovery and Classification in Amazon S3

1. Simple Introduction

Amazon Macie is a fully managed data security and data privacy service that uses machine learning and pattern matching to automatically discover, classify, and protect sensitive data stored in Amazon S3. It continuously monitors your S3 environment and generates detailed findings when it detects sensitive data (such as personally identifiable information) or security issues (such as publicly accessible buckets containing sensitive files).

PII (Personally Identifiable Information) includes names, email addresses, phone numbers, social security numbers, credit card numbers, passport numbers, medical records — any data that can identify a specific individual. Regulations like GDPR, HIPAA, and CCPA impose heavy fines for mishandling PII.

Analogy — The Intelligent Mail Sorter

Imagine your company receives thousands of physical documents (files in S3) daily. Without Macie, a security guard manually checks each document for sensitive data — impossible at scale. Macie is like hiring an AI-powered document scanner that reads every document automatically, stamps 'CONTAINS SSN' or 'CREDIT CARD DATA' on sensitive ones, and immediately alerts you if sensitive documents end up in an unlocked filing cabinet (public S3 bucket) — all without human effort.

Business Problem Macie Solves

Discover where sensitive data (PII, PHI, financial data) lives across thousands of S3 buckets

Alert when sensitive data is in publicly accessible or unencrypted buckets

Meet GDPR, HIPAA, CCPA, PCI-DSS data residency and discovery requirements

Understand your data exposure risk without manually scanning each file

Automate data classification — impossible to do manually at petabyte scale

2. Core Concepts & Terminology

Term	Definition
Sensitive Data Discovery Job	A Macie job that scans specified S3 buckets for sensitive data. Can run once or on a schedule
Managed Data Identifier	Pre-built ML + pattern detectors for 100+ sensitive data types (SSN, credit cards, passports, AWS credentials, etc.). Maintained by AWS
Custom Data Identifier	Your own regex + keyword rules to detect domain-specific sensitive data (e.g. employee ID format, internal project codes)
Sensitive Data Finding	An alert generated when Macie detects sensitive data in an S3 object. Includes: bucket, object, data type, severity, count
Policy Finding	An alert about S3 bucket security configuration issues: public access enabled, encryption disabled, cross-account access, replication to unmonitored account

Term	Definition
Finding Severity	LOW, MEDIUM, HIGH based on type of sensitive data and exposure risk
Sampling	For large buckets, Macie samples objects. Sampling depth configurable to balance cost vs coverage
Sensitive Data Types	Categories Macie detects: Credentials (AWS keys, passwords), Financial (credit cards, bank accounts), Health (PHI/medical), Personal (PII — SSN, DOB, passport), Network (IP ranges)
Allow List	Patterns or exact values to ignore during scanning (e.g. test data, sample SSNs in documentation)
Suppression Rule	Auto-archive findings matching criteria to reduce noise from expected patterns
Bucket Coverage	Overview of all S3 buckets in account/region: monitored, not monitored, sensitive data count, encryption status, public access status

3. Architecture & How Macie Works

Macie Sensitive Data Discovery Flow

1. ENABLE MACIE in AWS account (one click per region)
 - Macie gains read access to S3 bucket metadata and objects via service-linked role

2. AUTOMATIC BUCKET MONITORING (always on, free):

- Inventories ALL S3 buckets in the account/region continuously
- Evaluates bucket security: public access? encryption enabled? cross-account?
- Generates POLICY FINDINGS immediately when issues found

Example: 'Bucket prod-uploads is publicly readable' -> HIGH severity finding

3. SENSITIVE DATA DISCOVERY JOB (you create, has cost):

- Define: which buckets, which object types, which data identifiers, schedule
- Macie retrieves and analyses object content (text, CSV, JSON, Parquet, PDF, Excel, etc.)
- ML models + regex patterns scan each object for sensitive data
- Generates SENSITIVE DATA FINDINGS for matches

Example: 'File customer_export_2024.csv contains 4,521 SSNs and 892 credit card numbers'

4. FINDINGS DELIVERY:

- Console: Macie findings dashboard with severity breakdown
- Amazon EventBridge: auto-send findings to SNS, Lambda, Security Hub
- AWS Security Hub: aggregate with other security findings
- Amazon S3: export findings as JSON for SIEM ingestion

5. REMEDIATION ACTIONS (via EventBridge automation):

- Lambda triggered by finding: automatically block public access on bucket
- SNS notification to security team Slack channel
- Ticket created in Jira via Lambda

4. Key Features

Feature	Description	Business Value
100+ Managed Data Identifiers	Pre-built detectors for SSN, credit cards, passports, AWS keys, medical data, 80+ countries' PII formats	Immediate coverage for global compliance
Custom Data Identifiers	Regex + keyword rules for proprietary data formats	Detect business-specific sensitive data
Automatic Bucket Security	Continuously monitors all S3 bucket policies, ACLs, encryption status, public access	Instant alerts on misconfiguration
Multi-Account Support	Macie in AWS Organizations master account monitors all member account buckets	Centralised data security across org
Scheduled Jobs	Run discovery daily, weekly, monthly or one-time	Continuous compliance monitoring
Incremental Scanning	Only scan new/changed objects since last job run	Cost-efficient ongoing monitoring
Sensitive Data Map	Visual dashboard showing which buckets contain which sensitive data types	Data inventory and risk overview
Suppression & Allow Lists	Reduce false positives from known test data or internal patterns	Focus on real risks
Security Hub Integration	Findings appear in Security Hub alongside GuardDuty, Inspector, etc.	Unified security posture view
EventBridge Integration	Auto-trigger Lambda, SNS, Step Functions on findings	Automated remediation workflows
Finding Archive & Export	Export findings to S3 as JSON for long-term retention and SIEM	Compliance audit trail

5. Sensitive Data Types Macie Detects

Category	Examples Detected	Regulation
Personal PII	Full name + DOB + address, SSN (US), NI number (UK), SIN (Canada), Aadhaar (India)	GDPR, CCPA, PDPA
Financial Data	Credit card numbers (Visa/MC/Amex), bank account numbers, IBAN, routing numbers, tax IDs	PCI-DSS, GLBA
Health / Medical (PHI)	Medical record numbers, prescription drug names with patient context, NPI numbers, diagnosis codes	HIPAA
Credentials / Secrets	AWS access keys, private keys (RSA/PEM), passwords in plaintext, API keys, OAuth tokens	Security baseline
Network / Infrastructure	IP address ranges (internal subnets), MAC addresses	Infrastructure exposure
Government IDs	Passport numbers (40+ countries), driver's licence numbers, voter IDs	GDPR, national laws
Custom (Your Rules)	Employee ID formats, internal project codes, proprietary identifiers	Internal compliance

6. Real-World Use Cases

Industry	Use Case	Macie Feature
Healthcare (HIPAA)	Scan all S3 buckets for PHI monthly; alert if PHI found in non-compliant bucket	Scheduled jobs + PHI identifiers

Industry	Use Case	Macie Feature
Fintech (PCI-DSS)	Continuous monitoring for credit card data in S3; immediate alert + auto-quarantine on finding	EventBridge + Lambda remediation
Retail / E-Commerce (GDPR)	Discover customer PII across data lake S3 buckets; generate data inventory for GDPR Article 30 records of processing	Multi-bucket discovery + export
Healthcare Data Processor	Third-party data processor uses Macie to verify no patient data leaked into non-PHI S3 buckets before audit	Sensitive data finding reports
Enterprise IT	After S3 bucket misconfiguration incident, enable Macie across org via Organizations to get instant visibility	Multi-account centralised monitoring
Data Engineering	Data engineers accidentally include real PII in test datasets; Macie alerts before data reaches analytics pipeline	Custom identifiers + scheduled scan
M&A; Due Diligence	Scan acquired company's S3 buckets for sensitive data exposure risks before integrating into main org	One-time discovery job

7. AWS Service Integrations

AWS Service	Integration	Use Case
Amazon S3	Macie analyses S3 bucket configurations and object contents	Core data source
AWS Security Hub	Macie findings aggregated alongside GuardDuty, Inspector	Unified security dashboard
Amazon EventBridge	Findings automatically published as events for automation	Auto-remediation triggers
AWS Lambda	Triggered by EventBridge on finding: block public access, notify, quarantine	Automated response
Amazon SNS	Alert security team via SNS when HIGH severity finding occurs	Immediate human notification
AWS Organizations	Enable Macie for all member accounts from delegated admin account	Enterprise-wide coverage
AWS KMS	Macie findings stored encrypted with KMS	Finding data protection
Amazon CloudWatch	Metrics for discovery job status, finding counts	Operational monitoring

Chapter 4 — AWS Secrets Manager

Securely Store, Rotate & Retrieve Secrets — No More Hardcoded Credentials

1. Simple Introduction

AWS Secrets Manager is a fully managed service that helps you protect access to your applications, services, and IT resources — without the upfront investment and ongoing maintenance costs of operating your own secrets management infrastructure. It enables you to store secrets (database passwords, API keys, OAuth tokens, SSH keys), automatically rotate them, and retrieve them programmatically — so your application code never contains hardcoded credentials.

Analogy — The Digital Safe with a Key Rotation Robot

Imagine your app needs the password to the company database. The old way: write the password directly in the code (like writing your safe combination on a sticky note on the safe). Secrets Manager is a digital safe where you store passwords securely. Your app asks the safe for the password each time — and the safe verifies the app is authorised before revealing it. Better still: Secrets Manager has a robot that automatically changes the combination every 30 days and updates the safe — without you doing anything.

Business Problem Secrets Manager Solves

Hardcoded credentials in source code get accidentally committed to GitHub and stolen
Database passwords never get rotated because it requires downtime and coordination
No audit trail of which service accessed which credential and when
Secrets are copy-pasted into dozens of config files, becoming impossible to update
Compliance requires credential rotation (PCI-DSS: 90 days, HIPAA: as needed)

2. Core Concepts & Terminology

Term	Definition	Example
Secret	A piece of sensitive information Secrets Manager stores, encrypts, and manages	DB password, API key, TLS certificate, OAuth token
Secret Value	The actual sensitive data — stored as JSON key-value pairs or plain text	<code>{"username": "admin", "password": "s3cr3t!"}</code>
Secret Rotation	Automatically update a secret on a schedule by calling a Lambda function that changes the credential at the source	Rotate RDS password every 30 days
Rotation Lambda	A Lambda function (built-in templates for RDS/Redshift/DocumentDB or custom) that performs the actual credential change	Lambda updates RDS password and Secrets Manager simultaneously
Automatic Rotation	Rotation triggered by Secrets Manager on a cron schedule. Built-in support for RDS, Redshift, DocumentDB, and custom	Rotate every 30 days at midnight Sunday

Term	Definition	Example
Version	Each rotation creates a new secret version. Previous versions retained temporarily (AWSPREVIOUS stage) for rollback	AWSCURRENT = active, AWSPREVIOUS = last version
Staging Label	Tags on secret versions: AWSCURRENT (in use), AWSPENDING (being rotated), AWSPREVIOUS (recent previous)	App always requests AWSCURRENT
Resource Policy	JSON policy on a secret controlling cross-account access	Allow account 999 to read this secret
Replication	Replicate secret to multiple AWS regions for DR and global apps	Replicate DB credentials to all regions
Encryption	Secrets encrypted at rest with KMS (customer-managed or AWS managed key)	All secrets use AES-256 via KMS
VPC Endpoint	Access Secrets Manager API from private VPC without internet via PrivateLink	Lambda in private subnet retrieves secret
Parameter Store (SSM)	Similar AWS service — simpler, cheaper, less features (see comparison)	Non-sensitive config; sensitive config without rotation

3. Architecture & How Rotation Works

Secret Rotation Architecture — 4-Step Process

Secrets Manager uses a FOUR-STEP rotation Lambda pattern:

STEP 1: createSecret

Rotation Lambda creates a new credential at the target service (RDS, API provider)

New credential stored in Secrets Manager as AWSPENDING version

STEP 2: setSecret

Lambda sets the AWSPENDING credential on the target service

(e.g. creates new DB user with same permissions, or updates password)

STEP 3: testSecret

Lambda tests AWSPENDING credential actually works

(connects to DB, makes API call — verifies login succeeds)

STEP 4: finishSecret

Lambda promotes AWSPENDING -> AWSCURRENT

Old AWSCURRENT becomes AWSPREVIOUS (kept for rollback)

Rotation complete — zero application downtime

APPLICATION RETRIEVAL (every call):

```
app_code -> GetSecretValue(SecretId='prod/myapp/db', VersionStage='AWSCURRENT')
```

```
<- {'username': 'app_user', 'password': 'NewRotatedPassword!'}
```

Cache the secret in app memory with a TTL (e.g. 5 min) to reduce Secrets Manager API calls.

On cache miss OR on DB connection error -> re-fetch (handles rotation timing).

4. Key Features

Feature	Description	Business Value
Automatic Rotation	Built-in rotation for RDS, Redshift, DocumentDB, custom Lambda for others	No manual password changes ever
Native RDS Integration	Deep integration: Secrets Manager + Lambda + RDS — one checkbox enables auto-rotation	DB credentials rotate automatically
KMS Encryption	Every secret encrypted with KMS (customer-managed or AWS managed key)	Encrypted at rest always
Cross-Account Access	Resource policy allows other accounts to read the secret	Centralised secrets for multi-account orgs
Cross-Region Replication	Replicate secrets to multiple regions; replicas auto-update on rotation	Global apps and DR
Versioning	All versions retained with staging labels; rollback by requesting AWSPREVIOUS	Safe rotation with rollback capability
VPC Endpoint Support	PrivateLink endpoint — no internet needed to retrieve secrets	Private subnet access
Resource-Based Policy	Fine-grained control using JSON policy on the secret itself	Cross-account and service-level access
CloudTrail Auditing	Every GetSecretValue and rotation event logged	Full credential access audit trail
AWS SDK Integration	Native SDK support in Python, Java, Node.js, Go, .NET	Easy integration in any app
Secrets Tagging	Tag secrets for cost allocation and access control (ABAC)	Organised secret management
Recovery Window	Deleted secrets enter 7-30 day recovery window	Accidental deletion protection

5. Real-World Use Cases

Use Case	Configuration	Benefit
RDS Password Management	Enable rotation on RDS secret; Lambda rotates every 30 days; application uses GetSecretValue	Zero hardcoded DB passwords; automated rotation
API Key Storage	Store third-party API keys (Stripe, Twilio, SendGrid); Lambda retrieves at runtime; rotate manually when provider issues new keys	Centralised API key vault with audit trail
Container Secrets (ECS/EKS)	ECS task definition references Secrets Manager ARN; AWS injects as env var at task start; rotated secrets picked up on next task launch	No secrets in Docker images or K8s manifests
CI/CD Pipeline Secrets	GitHub Actions / CodeBuild retrieves deploy credentials from Secrets Manager via IAM role; never in repo	No secrets in CI/CD config files
Multi-Region SaaS	Primary secret in us-east-1; replicated to eu-west-1 and ap-south-1; rotation propagates to all regions	Global app with consistent credentials
Micro-services Architecture	Each microservice has its own secret with resource policy allowing only that service's IAM role	Least-privilege per service
PCI-DSS Compliance	Database credentials rotated every 90 days automatically; CloudTrail proves rotation occurred for audit	Automated compliance evidence

6. Secrets Manager vs SSM Parameter Store

Feature	Secrets Manager	SSM Parameter Store
Primary Purpose	Secrets with auto-rotation	Configuration + secrets (static)
Automatic Rotation	Yes — built-in + custom Lambda	No — manual only
Cost	\$0.40/secret/month + \$0.05/10K API calls	Free (Standard); \$0.05/advanced/month
Max Value Size	65 KB	4 KB (standard) / 8 KB (advanced)
Cross-Account	Yes (resource policy)	No (same account only)
Cross-Region Replication	Yes (built-in)	No
Versioning	Full versioning with staging labels	Limited (last 100 versions for advanced)
RDS Native Integration	Yes (one-click rotation)	No
Encryption	KMS always (mandatory)	KMS optional (SecureString tier)
Hierarchical Names	Flat with slash convention	Native hierarchy with path
Best For	Passwords, API keys, anything needing rotation	App config, feature flags, non-rotating credentials

Chapter 5 — AWS Shield

Managed DDoS Protection — Standard (Free) and Advanced (Enterprise)

1. Simple Introduction

AWS Shield is a managed Distributed Denial of Service (DDoS) protection service that safeguards applications running on AWS. DDoS attacks flood your infrastructure with traffic, overwhelming it until legitimate users cannot access your service. AWS Shield provides two tiers: Shield Standard (free, automatic for all AWS customers) and Shield Advanced (paid, enterprise-grade protection with 24x7 expert support).

A DDoS (Distributed Denial of Service) attack is when an attacker uses thousands of compromised machines (a botnet) to simultaneously send massive amounts of traffic to a target, exhausting its bandwidth, CPU, or connection capacity. These attacks can take down websites and cost thousands in unexpected AWS charges.

Analogy — The Nightclub Bouncer (Standard) vs. Armed Security Team (Advanced)

Standard Shield is like having a single bouncer at the door who spots and removes obvious troublemakers (known attack patterns). He handles most situations for free. Shield Advanced is like hiring a full private security team with intelligence analysts, surveillance cameras, direct police contact (DRT — DDoS Response Team), full incident investigation, and a guarantee that if anything gets through, the security company covers your damages (cost protection).

Business Problem Shield Solves

DDoS attacks can take your application completely offline within seconds

Massive traffic floods cause auto-scaling — resulting in enormous unexpected AWS bills

Attack traffic consumes bandwidth causing collateral slowness even before crashing

Small teams cannot manually respond to a large DDoS fast enough

Need SLA guarantees and expert help during an active sophisticated attack

2. Core Concepts & Terminology

Term	Definition
DDoS Attack	Distributed Denial of Service — attacker overwhelms target with traffic from many sources to make it unavailable
Volumetric Attack (Layer 3/4)	Floods bandwidth with huge volumes of UDP/ICMP packets. Measured in Gbps or Mpps. E.g. UDP flood, ICMP flood, DNS amplification
Protocol Attack (Layer 4)	Exploits weaknesses in Layer 3/4 protocols. E.g. SYN flood — sends thousands of half-open TCP connections to exhaust server state tables
Application Layer Attack (Layer 7)	Targets specific application functionality with seemingly legitimate HTTP requests. E.g. HTTP flood, Slowloris, GET/POST floods — not stopped by Shield alone; needs WAF
Attack Vector	The method used: volumetric (traffic volume), protocol (connection exhaustion), application (HTTP floods)

Term	Definition
Mitigation	Detecting and blocking attack traffic while passing legitimate traffic. Shield provides automatic mitigation
DRT (DDoS Response Team)	AWS expert team available 24x7 to Shield Advanced customers during active attacks
Cost Protection	Shield Advanced reimburses AWS charges (EC2, ALB, CloudFront, Route 53) incurred due to scaling during a DDoS attack
Health-Based Detection	Shield Advanced uses Route 53 health checks to detect attack impact on availability
Proactive Event Response	DRT proactively contacts Shield Advanced customers when they detect attacks before the customer notices
Attack Diagnostics	Shield Advanced provides detailed real-time and historical attack reports and metrics
SRT (Security Response Team)	AWS team that assists with incident response — available via AWS Support for Advanced customers

Shield Standard vs Shield Advanced

Feature	Shield Standard	Shield Advanced
Cost	FREE for all AWS customers	\$3,000/month per org (covers all resources)
Availability	Automatic, always-on	Must be subscribed and enabled
Attack Coverage	L3/L4 volumetric and protocol attacks	L3/L4 + L7 (with WAF) + sophisticated attacks
Attack Size	Common/known attack patterns	All sizes including nation-state-level attacks
DRT Access	No	Yes — 24x7 DDoS Response Team
Cost Protection	No	Yes — AWS credits back scaling costs from attacks
Attack Reports	Basic CloudWatch metrics	Detailed real-time + historical attack diagnostics
Health-Based Detection	No	Yes — Route 53 health check integration
Proactive Response	No	Yes — DRT contacts you proactively
Protected Resources	EC2, ALB, CloudFront, Route 53 (automatic)	Same + Global Accelerator + Elastic IP + granular config
SLA Guarantee	No	Yes — AWS guarantees mitigation
WAF Integration	Not included	AWS WAF included at no extra charge for protected resources

3. Architecture & How Shield Works

Shield Protection Architecture

INTERNET TRAFFIC -> AWS Edge Network (Shield Standard always active here)

LAYER 3 / 4 PROTECTION (Shield Standard + Advanced):

- Traffic enters via CloudFront, ALB, EC2 Elastic IP, or Route 53
- AWS global scrubbing centres detect anomalous traffic patterns
- Known DDoS signatures matched against continuously-updated threat database
- Volumetric floods (UDP/ICMP/SYN) absorbed and dropped at AWS edge
- Legitimate traffic passed through to your application
- Happens in milliseconds — transparent to your app

LAYER 7 PROTECTION (Shield Advanced + AWS WAF required):

- Shield Advanced detects application-layer attack patterns
- DRT works with you / proactively creates WAF rules to block attack traffic
- Rate-based WAF rules automatically block IPs sending too many requests
- CAPTCHA challenges issued to suspect traffic

SHIELD ADVANCED ATTACK RESPONSE WORKFLOW:

1. Attack begins: Shield detects anomaly via health checks and traffic analysis
2. Automatic L3/L4 mitigation activates within seconds
3. If L7: DRT notified; proactively contacts customer; creates WAF rules
4. Customer can call DRT directly via support channel
5. Post-attack: detailed report provided; cost protection credit applied
6. Architecture review if attack exposed weakness

4. Key Features

Feature	Tier	Description
Always-On Detection	Standard + Advanced	No manual configuration — protection is automatic from day one
Automatic L3/L4 Mitigation	Standard + Advanced	UDP floods, SYN floods, amplification attacks absorbed at AWS edge
AWS Network Scrubbing	Standard + Advanced	Traffic redirected through AWS scrubbing centres to filter attack traffic
DRT (DDoS Response Team)	Advanced only	24x7 expert team assists during active attacks via phone/email/chat
DDoS Cost Protection	Advanced only	Waive scaling charges (EC2, ALB, CloudFront, Route 53) caused by DDoS
WAF Integration	Advanced only	WAF usage free for protected resources; DRT creates WAF rules during attacks
Attack Diagnostics	Advanced only	Real-time attack dashboard, vectors, magnitude, geographic sources
Global Threat Intelligence	Standard + Advanced	Continuously updated from AWS-observed attack patterns across all customers
Proactive Engagement	Advanced only	DRT proactively contacts you based on health check violations during attacks

Feature	Tier	Description
Health-Based Detection	Advanced only	Route 53 health checks feed into Shield Advanced attack detection
Multi-Account Protection	Advanced only	Protect resources across all accounts in AWS Organization under one subscription
Global Accelerator Support	Advanced only	Protect Global Accelerator endpoints for non-HTTP workloads

5. Real-World Use Cases

Industry	Attack Scenario	Shield Tier & Configuration
E-Commerce	Competitor launches UDP flood on checkout page during peak sale event	Advanced + CloudFront + WAF rate limiting
Gaming	Rival guild DDoSes game server IP to disrupt tournament match	Advanced + Elastic IP protection + DRT on standby
Financial Services	Nation-state SYN flood targets banking portal during market hours	Advanced + DRT + NLB + proactive engagement enabled
News / Media	Hacktivist HTTP flood targets news site during breaking story	Advanced + CloudFront + WAF + L7 DRT rules
Healthcare	Ransomware group threatens DDoS unless ransom paid — infrastructure resilience needed	Advanced + Global Accelerator + DRT contact plan
Government / Public Sector	Election website targeted during voting period	Advanced + Route 53 + CloudFront + full DRT engagement
Startup	Basic volumetric attacks from script kiddies	Standard (free) + CloudFront is sufficient
SaaS Platform	Customer's competitor pays for DDoS-for-hire service targeting SaaS login page	Advanced + ALB + WAF rate-based rules

6. AWS Service Integrations

AWS Service	Integration	Purpose
Amazon CloudFront	Shield protects CloudFront distributions at edge	Absorb L3/L4 attacks before origin; CDN as DDoS buffer
AWS WAF	Shield Advanced + WAF: L7 protection; DRT creates WAF rules during attacks	Application-layer DDoS protection
Amazon Route 53	Shield protects Route 53 endpoints; health checks feed Shield Advanced detection	DNS-layer DDoS protection + health monitoring
AWS ALB / NLB	Shield Standard + Advanced protect load balancer endpoints	Protect entry point to application tier
Amazon EC2 (Elastic IP)	Shield Advanced explicitly protects Elastic IPs	Protect directly-exposed EC2 instances
AWS Global Accelerator	Shield Advanced protects GA endpoints for TCP/UDP workloads	Gaming, IoT, non-HTTP DDoS protection
Amazon CloudWatch	Attack metrics (DDoSDetected, DDoSAttackBitsPerSecond) available	Monitor attack volume and duration
AWS Firewall Manager	Centrally manage Shield Advanced policies across accounts	Org-wide DDoS protection governance

Chapter 6 — AWS WAF (Web Application Firewall)

Filter Malicious HTTP/HTTPS Traffic Before It Reaches Your Application

1. Simple Introduction

AWS WAF (Web Application Firewall) is a managed web application firewall that lets you monitor, filter, and block HTTP and HTTPS requests to your web applications based on conditions you define. It protects at the application layer (Layer 7) against common web exploits — things like SQL injection, cross-site scripting (XSS), bot traffic, and API abuse — which network firewalls (Shield) cannot stop because the attack traffic looks like legitimate HTTP requests.

WAF sits in front of your application (CloudFront, ALB, API Gateway, AppSync) and inspects every incoming HTTP request. For each request, WAF evaluates your rules and decides: ALLOW it through, BLOCK it (return 403), or COUNT it (for monitoring). A CAPTCHA challenge can also be issued to suspected bots.

Analogy — The Intelligent Customs Agent

Without WAF, traffic flows straight into your application like an airport with no customs — everyone walks through unchecked. AWS WAF is a highly trained customs agent who inspects every passenger (HTTP request): checks their passport (IP address), looks in their luggage (request body and parameters), checks the banned list (managed rules), and decides: let them through, detain them (CAPTCHA), or turn them away (BLOCK). Legitimate travellers pass without noticing the check.

Business Problem WAF Solves

SQL injection attacks steal data from databases via vulnerable web forms

XSS attacks inject malicious scripts into web pages served to other users

Bots scrape pricing, content, and credentials at scale, degrading performance

Credential stuffing attacks try thousands of stolen username/password pairs

API abuse — scrapers and competitors hammer your API with excessive requests

Zero-day exploits targeting specific application vulnerabilities before you can patch

2. Core Concepts & Terminology

Term	Definition	Example
Web ACL (Access Control List)	The main WAF resource. Contains an ordered list of rules. Attached to CloudFront, ALB, API Gateway, or AppSync	MyAppWebACL attached to my-alb
Rule	A condition + action pair: if request matches condition, take this action (Allow/Block/Count/CAPTCHA)	If IP in blacklist -> Block

Term	Definition	Example
Rule Group	A reusable collection of rules. Can be AWS Managed, marketplace, or custom	AWSManagedRulesCommonRuleSet
Managed Rule Group	Pre-built rule sets maintained by AWS or partners covering OWASP Top 10, bot detection, specific CVEs	AWSManagedRulesSQLiRuleSet
Statement	The condition part of a rule: what to inspect and how to match	Body contains 'UNION SELECT', IP matches 192.0.2.0/24
Action	What to do when a rule matches: Allow, Block (403), Count (monitor), CAPTCHA, Challenge	Block returns custom 403 error page
Default Action	What to do if NO rules match — Allow (whitelist mode) or Block (default deny mode)	Default Allow is most common
WCU (Web ACL Capacity Units)	Measure of rule complexity. Each Web ACL has 5,000 WCU limit	Simple IP match = 1 WCU; regex body match = 10 WCU
Rate-Based Rule	Block IPs that send more than N requests in 5 minutes	Block any IP sending >2,000 req/5min
IP Set	A reusable list of IP addresses or CIDR ranges to allow or block	Office IP whitelist, known bad actor blacklist
Regex Pattern Set	Reusable set of regular expressions for matching request components	Match malicious user-agent strings
Bot Control	AWS Managed Rule Group using machine learning to detect and manage bots (common bots, targeted bots)	Block scrapers; allow Googlebot
Fraud Control (ATP / ACFP)	Account Takeover Prevention (stolen credentials) and Account Creation Fraud Prevention	Detect credential stuffing on login page
Label	WAF can add labels to matching requests; subsequent rules can match on labels (cascading rules)	Label 'verified-bot' -> different rule applies
Logging	Full request logs (headers, body, matched rule) to Kinesis Firehose, S3, or CloudWatch Logs	Store all requests for forensic analysis
Scope	CLOUDFRONT (global, us-east-1 only) or REGIONAL (ALB, API GW, AppSync — per region)	Create WAF in us-east-1 for CloudFront

3. Architecture & How WAF Evaluates Requests

WAF Rule Evaluation Flow

HTTP REQUEST arrives at CloudFront / ALB / API Gateway

|

v

[AWS WAF Web ACL evaluation]

Rules evaluated in ORDER (lowest priority number first):

Rule 1 (Priority 1): IP Reputation List (AWS Managed)

-> Is source IP on known bad-actor list?

-> YES: BLOCK immediately (skip remaining rules) -> 403 Forbidden

-> NO: continue to Rule 2

Rule 2 (Priority 2): Geo-Block (custom rule)

-> Is request from North Korea or sanctioned country?

-> YES: BLOCK -> NO: continue

Rule 3 (Priority 3): Rate Limit (rate-based rule)

-> Has this IP sent >1000 requests in last 5 minutes?

-> YES: BLOCK -> NO: continue

Rule 4 (Priority 4): SQL Injection (AWS Managed SQLi rule)

-> Does request body/query/headers contain SQL injection patterns?

-> YES: BLOCK -> NO: continue

Rule 5 (Priority 5): XSS (AWS Managed XSS rule)

-> Does request contain XSS patterns?

-> YES: BLOCK -> NO: continue

Rule 6 (Priority 6): Bot Control (AWS Managed)

-> Is this automated bot traffic?

-> YES (unwanted): BLOCK | YES (Googlebot): ALLOW -> NO: continue

No rules matched -> DEFAULT ACTION: ALLOW

|

v

Request forwarded to APPLICATION (origin)

LOGGING: all matched/unmatched rules + full request logged to Firehose/S3

4. AWS Managed Rule Groups

Rule Group	Protects Against	WCU
AWSManagedRulesCommonRuleSet	OWASP Top 10: SQLi, XSS, LFI, RFI, path traversal, HTTP protocol violations	700
AWSManagedRulesSQLiRuleSet	SQL injection patterns in query strings, body, cookies, URI, headers	200
AWSManagedRulesKnownBadInputsRuleSet	Log4Shell (CVE-2021-44228), Spring4Shell, SSRF patterns, null bytes, Java deserialization	200

Rule Group	Protects Against	WCU
AWSManagedRulesIPReputationList	IPs associated with bots, scrapers, DDoS botnets, TOR exit nodes	25
AWSManagedRulesAnonymousIPList	VPN services, proxies, Tor, hosting providers used to hide attacker identity	50
AWSManagedRulesAmazonIPReputationList	Amazon-curated threat intelligence from AWS-observed attacks	25
AWSManagedRulesLinuxRuleSet	Linux-specific attacks: LFI, command injection targeting Linux/Unix paths	200
AWSManagedRulesWindowsRuleSet	Windows-specific attacks: PowerShell injection, Windows command injection	200
AWSManagedRulesPHPRuleSet	PHP injection, insecure PHP function calls, PHP-specific exploits	100
AWSManagedRulesWordPressRuleSet	WordPress-specific exploits, wp-admin attacks, XML-RPC abuse	100
Bot Control — Common	Detect and manage high-volume bots: scrapers, crawlers, content theft (ML-based)	50 + 25/rule
Bot Control — Targeted	Sophisticated bots using browser impersonation, CAPTCHA evasion (JS challenge + ML)	Variable
ATP (Account Takeover Prevention)	Credential stuffing: anomalous login patterns, stolen credential use, token theft	Variable
ACFP (Account Creation Fraud)	Fake account creation bots, referral fraud, promotion abuse	Variable

5. Key Features

Feature	Description	Business Value
AWS Managed Rule Groups	Pre-built, continuously updated rule sets for OWASP, CVEs, bots	Instant protection without writing rules
Custom Rules	Write your own match conditions on any request component	Business-specific blocking logic
Rate-Based Rules	Auto-block IPs exceeding request rate thresholds	API abuse and DDoS Layer 7 protection
Bot Control	ML-powered bot detection — allow good bots, block bad ones	Protect content and prevent scraping
Account Takeover Prevention	Detect stolen credentials on login endpoints	Prevent account takeover attacks
CAPTCHA & Challenge	Issue browser JS challenge or visual CAPTCHA to suspicious requests	Bot filtering without full block
Geo-Blocking / Geo-Allow	Allow or block by country or continent	Compliance, licensing, fraud prevention
IP Reputation Lists	AWS and third-party managed threat intelligence IP lists	Block known malicious IPs automatically
Logging to Firehose / S3	Full request log with matched rule, IP, URI, headers, body sample	Forensic analysis and compliance

Feature	Description	Business Value
CloudWatch Metrics	Per-rule request counts for allowed, blocked, counted requests	WAF rule effectiveness monitoring
Firewall Manager Integration	Deploy WAF policies across all accounts/regions from central account	Org-wide WAF governance
Version Control for Managed Rules	Pin to specific version; test new version before switching	Controlled managed rule updates
Labels	Tag requests with labels as they flow through rules for cascading logic	Complex multi-condition rule logic
Payload Inspection	Inspect request body up to 8 KB (configurable up to 64 KB)	Catch attacks hidden in POST body

6. Real-World Use Cases

Industry	Attack Scenario	WAF Configuration
E-Commerce	SQL injection via product search box; XSS in review submission; price scraping bots	CommonRuleSet + SQLiRuleSet + Bot Control
Banking / Fintech	Credential stuffing on login page; API abuse on transaction endpoints; DDoS L7	ATP + rate-based rules + IP reputation + Shield Advanced
Healthcare API	SSRF attacks probing internal services via patient API; LFI on document endpoint	KnownBadInputsRuleSet + LinuxRuleSet + geo-restrict
SaaS Platform	Competitor scraping customer data; fake account creation for trial abuse	Bot Control Targeted + ACFP + rate limiting
Gaming	Cheating bots automating game actions; fake account farming	Bot Control + custom JS challenge rules
News / Media	Competitor scrapes article content; piracy bots download paid content	Bot Control Common + IP reputation + rate limiting
Public API	Aggressive API consumers causing cost spikes; undocumented scraping	Rate-based rules per URI path + API key header check
Government	Nation-state probing for vulnerabilities in citizen portal; geo-block all non-domestic IPs	All managed rule groups + geo-allow list

7. AWS Service Integrations

AWS Service	Integration	Use Case
Amazon CloudFront	Attach Web ACL to distribution — rules evaluated at edge globally	Global L7 protection at CDN layer
AWS ALB	Attach Web ACL to Application Load Balancer — regional protection	Protect web apps / APIs behind ALB
Amazon API Gateway	Attach Web ACL to REST or HTTP API stage	Protect REST APIs from abuse and injection
AWS AppSync	Attach Web ACL to GraphQL API	Protect GraphQL from introspection abuse and injection
AWS Shield Advanced	WAF included free with Shield Advanced; DRT creates WAF rules during L7 DDoS	Combined L3/L4/L7 DDoS protection
AWS Firewall Manager	Centrally deploy and manage WAF Web ACLs across org accounts	Enterprise WAF governance

AWS Service	Integration	Use Case
Amazon Kinesis Data Firehose	WAF logs streamed to Firehose for delivery to S3 or SIEM	Security analytics and compliance logging
Amazon CloudWatch	Per-rule allow/block/count metrics for dashboards and alarms	Real-time WAF monitoring
AWS Lambda	Custom Lambda authorizer can use WAF decisions; Lambda@Edge alongside CloudFront WAF	Advanced custom rule logic
AWS Security Hub	WAF findings published to Security Hub	Unified security visibility

8. Master Summary — All 6 Security Services

Service	Protects	What It Does	Key Compliance
AWS IAM	Who accesses AWS	Authentication and authorisation for all AWS API calls	All compliance standards
AWS KMS	Data at rest	Create/manage encryption keys; encrypt data across AWS services	HIPAA, PCI-DSS, GDPR, SOC2
Amazon Macie	Sensitive data in S3	Discover and classify PII, PHI, credentials in S3 buckets	GDPR, HIPAA, CCPA, PCI-DSS
Secrets Manager	Credentials and secrets	Store, rotate, and retrieve passwords/API keys without hardcoding	PCI-DSS, HIPAA, SOC2
AWS Shield	Infrastructure from DDoS	Absorb L3/L4 DDoS attacks; Advanced adds L7 + DRT support	Availability SLAs
AWS WAF	Applications from L7 attacks	Filter HTTP/HTTPS traffic: block SQLi, XSS, bots, rate abuse	PCI-DSS 6.6, OWASP Top 10

Defence-in-Depth: How All 6 Services Layer Together

LAYER 1 — IDENTITY (IAM)

Only authorised identities can access AWS resources. Roles, least-privilege, MFA.

LAYER 2 — NETWORK EDGE (Shield + CloudFront)

DDoS traffic absorbed at AWS edge before reaching your infrastructure.

LAYER 3 — APPLICATION LAYER (WAF)

Malicious HTTP requests (SQLi, XSS, bots) blocked before hitting your app servers.

LAYER 4 — ENCRYPTION (KMS)

All data stored encrypted. Even if storage is breached, data is unreadable without key.

LAYER 5 — SECRETS (Secrets Manager)

No credentials in code. DB passwords rotate automatically. Audit trail on every access.

LAYER 6 — DATA DISCOVERY (Macie)

Know where sensitive data lives. Alert on misconfigurations before a breach occurs.

RESULT: An attacker must defeat ALL 6 layers to successfully compromise your application.

This is Defence-in-Depth — the gold standard of AWS security architecture.